

二维数组

int a[6]

1	2	3	4	5	6
---	---	---	---	---	---

一维数组

int a[4][4]

1	2	4	1
2	1	5	2
6	3	7	9
3	5	4	2

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]
a[3][0]	a[3][1]	a[3][2]	a[3][3]

二维数组： 数组名[常量表达1][常量表达2]

int a[4][5]：定义了一个4行5列的整形数组，
下标是a[0][0]—a[3][4].

数组中元素a[2][5]表示？

初始化

一、批量赋值。

$a[2][2] = \{1, 2, 3, 4\};$

1	2
3	4

赋值时**先行后列**。一行赋值完后跳到下一行第一列。

8	7	
5	9	

该怎么赋值？

$a[3][3] = \{8, 7, 0, 5, 9, 0, 0, 0, 0\};$

$a[3][3] = \{\{8, 7\}, \{5, 9\}\};$

二维数组可以看成是由多个一维数组组成的：

$a[3][4]$ 可以看成由3个 $a[4]$ 数组组成或者4个 $a[3]$ 数组组成

初始化

二、循环赋值。

1	2
3	4

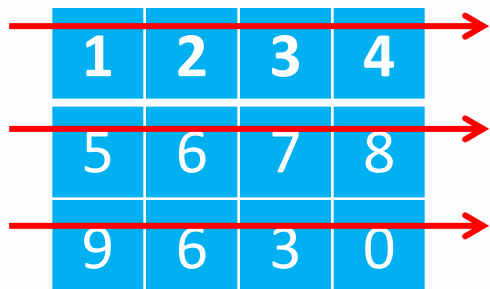
```
for(int i=0;i<=1;i++)
{
    for(int j=0;j<=1;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
```

8	7	
5	9	

```
for(int i=0;i<=1;i++)
{
    for(int j=0;j<=1;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
```

遍历

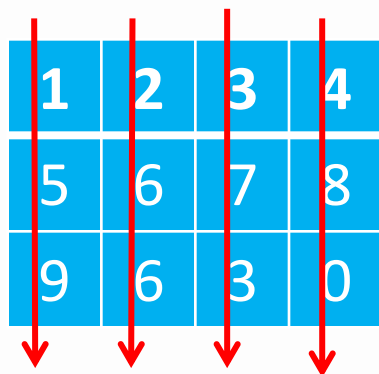
一、按行遍历



A 3x4 grid of blue squares containing the numbers 1 through 8, and 9, 6, 3, 0 in the third row. Three red arrows point horizontally from left to right across each row, indicating the direction of row-major traversal.

1	2	3	4
5	6	7	8
9	6	3	0

二、按列遍历



A 3x4 grid of blue squares containing the numbers 1 through 8, and 9, 6, 3, 0 in the third row. Four red arrows point vertically from top to bottom in each column, indicating the direction of column-major traversal.

1	2	3	4
5	6	7	8
9	6	3	0

```
for(int i=1;i<=3;i++)//行
{
    for(int j=1;j<=4;j++)//列
    {
        scanf("%d",&a[i][j]);
    }
}
for(int j=1;j<=4;j++)//列
{
    for(int i=1;i<=3;i++)//行
    {
        scanf("%d",&a[i][j]);
    }
}
```

无论是哪种遍历方法， $a[i][j]$ 都是表示数组中第*i*行第*j*列的数，这是固定不变的。

存储方式

a[3][4]

1	2	3	4
5	6	7	8
9	6	3	0

在计算机内存中，所有的数据都是以线性的方式存储的！！
不存在多维存储！！

第一行

第二行

第三行

	1	2	3	4	5	6	7	8	9	6	3	0	
--	---	---	---	---	---	---	---	---	---	---	---	---	--

a[0][0] a[0][1] a[0][2] a[0][3] a[1][0] a[1][1] a[1][2] a[1][3] a[2][0] a[2][1] a[2][2] a[2][3]

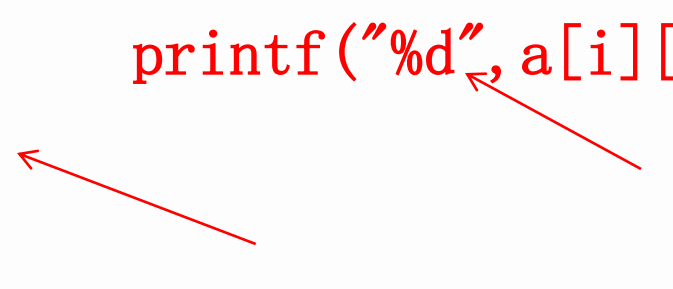
在计算机内存中，存储的方式是**先行后列**，存储方式和数的**全排列**比较类似，所以我们在平时学习过程中，一般也是按照**先行后列**的方式来遍历数组的。

111	112	113
121	122	123
131	132	133
211	212	213
221	222	223

输出

二、循环输出。

```
for(int i=0;i<m;i++)  
{  
    for(int j=0;j<n;j++)  
    {  
        printf("%d", a[i][j]);  
    }  
}
```



如果要按照矩阵的样式输出，
该如何添加空格和回车？

```
for(int i=0;i<n;i++)  
printf("%d", a[i][m-1]);
```

如果只需要输出第m
列的所有数据呢？

输出第i行第 (m-1) 列的所有数据。

矩阵求和

输入一个 $n*m$ 的矩阵，求该矩阵中所有元素之和

输入：

3 3

1 2 3

3 4 7

5 6 8

输出：

39

$n*m$ 的矩阵，一定是 n 行 m 列

矩阵最大值

输入一个 $n*m$ 的矩阵，求该矩阵中最大的元素和最小的元素，分别输出最大元素的行列和最小元素的行列号，如果有多个，输出行号最小的那一个，如果行号相同，输出列号最小的那一个。

输入：

```
3 3  
1 2 3  
3 4 7  
5 6 8
```

输出：

```
3 3  
1 1
```

$n, m \leq 100;$

矩阵变换

输入一个 $n*n$ 的矩阵，交换矩阵的行和列，变成一个新的矩阵。

输入：

3

1 2 3

3 4 7

5 6 8

输出：

1 3 5

2 4 6

3 7 8

提示：交换矩阵的行和列就是把矩阵的第 i 行变成矩阵的第 i 列，只能用于 $n*n$ 矩阵。

$n \leq 100$;

题 解

1	2	3		1	3	5
3	4	7	→	2	4	6
5	6	8		3	7	8

原矩阵中5是第三行第一列的数，
现在5是第一行第三列的数。

原矩阵中7是第二行第三列的数，
现在7是第三行第二列的数。

源代码

```
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j++)
    {
        scanf("%d",&a[i][j]);
    }
}
for(int j=1;j<=n;j++) //先循环列
{
    for(int i=1;i<=n;i++) //再循环行
    {
        printf("%d ",a[i][j]);
    }
    printf("\n");
}
```

a[j][i]表示第j行第i列

数组

数组不仅仅是用来存储某个位置的数字，也可以用来存储**某种状态**，比如校门外的树，还可以用来**表示区间**，数组的含义是我们手动赋予的，含义可以多种多样，不要仅仅局限于存储某个位置的数。

输入一个长度为 n 的整型数列，先给出 m 次查询 k ， k 表示数列中前 k 个数之和，并且输出结果。

输入：

6

1 2 3 4 5 6

3

2 4 5

输出：

3

10

15

$n \leq 100000$, $m \leq 200000$, 输入的数在int范围内

解题思路

解法一：

对于每一次查询 k ，计算 $a[i]—a[k]$ 之和，并且用一个数组来存储。

时间复杂度： $O(m*n)$ 。（很慢）

解法二：

前面第一种解法造成了很多次计算浪费，比如，第一次查询 $k=n$ ，这样就会计算出 $a[1]—a[n]$ 的区间之和，在计算的过程中，肯定会逐步计算出 $a[1]—a[i]$ 的区间和。我们下次查询的时候又会计算一次，很浪费时间。

解题思路

解法二：

(1) 计算出所有的前缀和，即计算出所有的 $a[1] \cdots a[k]$ 之和，并且用一个数组来存储 $sum[i]$ 表示 $a[1] \cdots a[i]$ 的区间之和。

$$sum[i] = sum[i-1] + a[i];$$

(2) 对于每一次查询 k ，只需要找到对应的 $sum[k]$ ，即为需要查询的结果。

时间复杂度 $O(m+n)$ 。（很快）

注意：在程序设计中，对于存储“求和”这一类的最好都用`long long`来存储，以免数据范围不够造成溢出。

前缀之和

输入一个长度为 n 的整型数列，现给出 m 次查询 (x, y) ， (x, y) 表示数列中 $[x, y]$ 区间内所有数之和，并且输出结果。（起点从1开始）

输入：

6

1 2 3 4 5 6

3

1 3

1 5

3 4

输出：

6

15

7

样例说明：1 3表示第1个数到第3个数之和

$n \leq 3000, m \leq 200000$

计算鞍点

求一个 $N*N$ 的矩阵中的马鞍数，所谓的马鞍数是指在行上最大而在列上是最小的数，如果有，输出他的行和列，如果没有输出“not found”。

输入：

4

11 3 5 6

12 4 7 8

10 5 6 9

8 6 4 7

输出：

4 1

输入：

4

11 3 5 6

12 4 7 8

10 5 6 9

7 6 4 8

输出：

not found

样例1解释：

第一行最大值11，第二行最大值12，
第三行最大值10，第四行最大值8，
四个最大值中只有8是所在列最小的

$n \leq 100$

解题思路

第一步：找出每一行的最大值，并且记录它的位置（行和列）

第二步：判断每一行的最大值在这一列是否是最小的，如果是，输出行和列，输出结果，如果不是继续找下一行。

第三步：很明显，鞍点最多只有一个，所以找到一个就马上结束，如果找完都没有找到，那就没有鞍点。

杨辉三角形

杨辉三角形的格式如下：

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
```

.....

为了方便存储：

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

请仔细观察杨辉三角形，并且找到他的规律

NOIP数论常见考点！！！！

杨辉三角形

已知杨辉三角形，输入一个正整数k，请输出杨辉三角形第k行的所有数据，每个数据之间用一个空格隔开。

输入：

5

输出：

1 4 6 4 1

$K \leq 50$

已知杨辉三角形，输入一个正整数n，请输出杨辉三角形第n个数是多少。

输入：

20

输出：

5

$n \leq 1000$

二项式定理

$$(x+y)^1 = 1x + 1y;$$

$$(x+y)^2 = 1x^2 + 2xy + 1y^2;$$

$$(x+y)^3 = 1x^3 + 3x^2y + 3xy^2 + 1y^3;$$

$$(x+y)^4 = 1x^4 + 4x^3y + 6x^2y^2 + 4xy^3 + 1y^4;$$

$$(x+y)^5 = 1x^5 + 5x^4y + 10x^3y^2 + 10x^2y^3 + 5xy^4 + 1y^5;$$

1					
1	1				
1	2	1			
1	3	3	1		
1	4	6	4	1	
1	5	10	10	5	1

杨辉三角形的应用一：二次项定理

二项式定理

给定一个多项式 $(x+y)^k$ ，求出 $x^n y^m$ 的系数 ($n+m=k$)。

输入：

2 3

输出：

10

输入：

1 2

输出：

3

$n, m \leq 20$.